

Trivia Night

STUDENT HANDOUT · ACTIVITY 2 · PYTHON PROGRAMMING

Your mission: Build a console trivia game that asks questions, checks answers, keeps score, and responds politely when input is missing. Use AI to help create and review trivia content; write and understand the Python that runs the game.

Textbook Connection

Course textbook: *Murach's Python Programming*, Section 2, Parts A and B. The course materials align this activity with syllabus Weeks 4–5. Part A covers functions, lists, tuples, and exceptions; Part B covers dictionaries, strings, and dates/times.

The course materials identify this reading by section and part, not by numbered chapter. Chapter numbering can differ by edition, so use the table of contents in your assigned edition to locate Section 2, Parts A and B.

What You Will Practice

- Organize related information using lists and dictionaries.
- Write and call a function that accepts information and returns a result.
- Repeat work with a `for` loop and track a running score.
- Clean text with string methods before comparing answers.
- Handle blank answers and end-of-input without an unfriendly crash.
- Ask AI for structured content, verify it, and use AI as a learning partner.

1. Store Questions as Data

A dictionary stores details about one question. A list stores multiple questions, so the same game logic can work with every item in the question bank. Here are two question dictionaries inside one list:

```
questions = [  
    {  
        "question": "What is the capital of Japan?",  
        "answer": "Tokyo",  
        "category": "Geography",  
    },  
    {  
        "question": "Which planet is known as the Red Planet?",  
        "answer": "Mars",  
        "category": "Space",  
    },  
]
```

Each set of braces is one dictionary, and the comma separates the two dictionaries in the list. The labels such as "question" and "answer" are **keys**; the information beside each label is its **value**. Use square brackets and a key to look up a value: `questions[0]["answer"]` gives "Tokyo". List positions start at 0, so

`questions[1]["question"]` gives the second question.

Term	Meaning	Example
Variable	A name that refers to a value.	<code>score = 0</code>
List	An ordered collection that can change.	<code>questions = [q1, q2]</code>
Dictionary	Key/value pairs describing an item.	<code>{"answer": "Tokyo"}</code>
Key / value	A dictionary label / the information stored under it.	<code>"answer" / "Tokyo"</code>
Tuple	An ordered collection not meant to change.	<code>options = ("A", "B", "C")</code>

Every question needs the same three keys: "question", "answer", and "category". A missing or misspelled key causes an error when the program tries to use it.

2. Functions: Give One Job a Name

A **function** is a named, reusable block of code. A **parameter** is information the function receives. A **return value** is the result sent back to the code that called the function.

This small function answers a yes/no check. Calling it with different values reuses the same instructions:

```
def is_ready(answer):
    return answer.strip().lower() == "yes"

first_check = is_ready(" YES ") # True
second_check = is_ready("not yet") # False
```

Here, `answer` is a parameter. `return` sends the comparison result back to the caller. A second useful pattern is to give a function a dictionary and have it display that record:

```
def show_question(question):
    print(f"[{question['category']}] {question['question']}")

show_question(questions[0])
show_question(questions[1])
```

The same function can display either question because the dictionary is passed in as the parameter. In your game, `ask_question(question)` should ask one question and return `True` for a correct answer or `False` otherwise; `main()` can use that result to decide whether to add a point.

3. Text Input: Normalize Before Comparing

`input()` gives your program text. Clean both sides of a comparison so capitalization and extra spaces do not matter. The examples below show different typed answers being normalized to the same stored answer:

- `.strip()` removes spaces at the beginning and end.
- `.lower()` changes letters to lowercase.
- `.title()` changes words to title case; useful for display, but usually unnecessary for answer comparison.

```
correct_answer = "Tokyo"

typed_first = "  ToKyO  "
typed_second = "tokyo"

print(typed_first.strip().lower() == correct_answer.strip().lower()) # True
print(typed_second.strip().lower() == correct_answer.strip().lower()) # True
```

For the Mars question, the same comparison pattern also accepts " MARS " when the stored answer is "Mar s". Clean both the player's answer and the stored answer when comparing. Keep the question-bank data unchanged.

4. Input Errors: Blank Text Is Not an Exception

A **run-time error** is a problem that happens after Python has started running your program. For example, asking Python to divide by zero raises an exception while the program runs. If that exception is not handled, Python prints an error message and stops the program. A **syntax error** is different: it means Python cannot understand how the code is written, so it cannot start running that code.

```
# Run-time error: Python cannot divide a number by zero
result = 10 / 0
```

An **exception** is the signal Python raises for a run-time problem. `try` runs code that might raise an exception; `except` handles a particular exception so the program can respond instead of stopping abruptly. Handling an exception does not mean the problem never happened; it means your code chose what to do about it.

Important: pressing Enter without typing returns an empty string. That is valid input, not an exception, so check it with `if`. If the input stream closes, `input()` raises `EOFError`, which can be handled with `try/except`.

Example 1: pressing Enter produces blank text. An `if` check handles that ordinary input:

```
answer = input("Your answer: ")
if answer.strip() == "":
    print("Please enter an answer next time. This one is incorrect.")
```

Example 2: if input reaches end-of-file, handle the specific `EOFError`. After that, the blank check can still handle the empty answer consistently:

```
try:
    answer = input("Your answer: ")
except EOFError:
```

```
print("No input received. This answer will count as incorrect.")
answer = ""

if answer.strip() == "":
    print("A blank answer will count as incorrect.")
```

On Windows, end-of-file in a terminal is usually Ctrl+Z followed by Enter; on macOS or Linux, it is usually Ctrl+D. For this game, an unanswered question should not earn a point. Avoid a bare `except :`; catch the specific error you expect.

5. Loops and Score

A **loop** repeats code. A `for` loop can do something once for each item in a collection:

```
for item in questions:
    print(item["question"])
```

A **running total** is a variable updated as the program goes. Start the score at zero; add one only when the answer is correct. The built-in `len()` function tells you how many questions are in the list.

```
score = 0
score = score + 1 # award one point
```

An **f-string** inserts values into text. Put `f` before the quote and variable names inside braces:

```
player = "Sam"
score = 3
print(f"{player}, your score is {score}.")
```

Build It in Small Steps

TODO 1: Create the Question Bank

- Make `QUESTIONS` a list containing at least five dictionaries.
- Give every dictionary the three required keys.
- Choose questions with clear, checkable answers. Keep stored answers concise so matching is fair.
- Use AI for ideas or structured data, then verify every fact yourself.

TODOs 2–3: Ask and Check One Question

- Notice how the starter displays the category and question.
- Get the player's answer with `input()`.
- Compare cleaned, lowercase versions of the typed and stored answers.
- Return a Boolean: `True` for a match, `False` otherwise.
- Test a correct answer with different capitalization and extra spaces, then test a wrong answer.

TODO 4: Handle Missing Input

- Check whether the answer is blank after stripping whitespace.
- Use `try/except` for a real input exception such as `EOFError`.
- Print a friendly message and make sure an unanswered question earns no point.

TODO 5: Play the Full Game

- Loop through `QUESTIONS` with a `for` loop.
- Call `ask_question()` once per question.
- Print feedback and increase the score only for a correct answer.
- Print points earned out of questions asked using an f-string.

Work independently: make one small change, save, and run the program. Get one question working before testing the whole question bank.

Use AI as a Learning Partner

AI can make factual mistakes and can produce Python that looks convincing but does not run. You are responsible for checking the questions and understanding the code you submit.

1. Ask for one specific kind of help.
2. Inspect the response: does it have the requested structure, and are its facts correct?
3. Add or adapt a small part yourself.
4. Run your program and test it.
5. If it fails, share the exact error and ask for an explanation before requesting a fix.

Prompt for Question Content

```
Suggest five beginner-friendly trivia questions about [topic]. Return only valid Python data: a list of
```

Check that the response is valid Python data before using it. Independently verify every answer with a trustworthy source; an AI-generated source list is not proof that an answer is correct.

Prompts for Learning and Debugging

```
I am learning Python functions and loops. Explain what this code does one step at a time. Do not
```

```
This input behaves differently than I expected. Explain the error in beginner-friendly language and
```

```
Review this try/except block for input edge cases. Explain which cases are exceptions and which n
```

Do not share passwords, private information, or anything you should not share with an AI tool. Follow your instructor's rules about which tools you may use.

Test Before You Finish

- The question bank has at least five entries, and each has all three required keys.
- The game asks every question in the list.
- A correct answer earns one point; a wrong answer earns none.
- Capitalization and extra spaces do not make a correct answer wrong.
- A blank answer is handled politely and earns no point.
- The final score is correct and includes the number of questions.
- You can explain what your function returns and how the loop updates the score.

If something goes wrong, read the full error message from the bottom upward. Check spelling, quotation marks, parentheses, indentation, and dictionary keys. Test one question or input case at a time.

Stretch Goals

- Store fixed multiple-choice options as a tuple, for example `options = ("A", "B", "C")`.
- Let the player choose a category and play only matching questions. A **filter** selects items that meet a condition.

Optional Date and Time: Timestamp the Score

A **timestamp** is a date and time recorded for an event. Python's `datetime` tools can get the current date and time from the computer's clock. The starter already has `from datetime import datetime` at the top, so you do not need to add another import.

```
finished_at = datetime.now()
print(f"Game finished at: {finished_at}")
```

`datetime.now()` gives you a date-and-time value. Printing it directly includes seconds and may include extra precision. To choose a shorter display, use `strftime()`, which formats the value as text. In these format codes, `%Y` means four-digit year, `%m` month, `%d` day, `%H` hour (24-hour clock), and `%M` minute.

```
finished_at = datetime.now()
timestamp = finished_at.strftime("%Y-%m-%d %H:%M")
print(f"Game finished at: {timestamp}")
```

The displayed time comes from the computer running the game, so it depends on that computer's clock and time zone. This feature is optional; finish and test the main trivia game first.

Quick Vocabulary Review

Term	Meaning
Function	Reusable code that can accept parameters and return a result.

Term	Meaning
Parameter / return value	Information passed into a function / result sent back to its caller.
List / dictionary / tuple	Ordered changeable collection / key-value collection / ordered unchangeable collection.
Boolean	One of two values: <code>True</code> or <code>False</code> .
Run-time error	A problem that happens while the program is running; an unhandled exception can make the program stop.
Exception	A signal Python raises for a problem during execution; <code>try/except</code> can handle it.
String method	An operation on text, such as <code>.strip()</code> or <code>.lower()</code> .
f-string	A string that can include values inside braces, such as <code>f"Score: {score}"</code> .
<code>datetime</code>	Python's date/time tools; <code>datetime.now()</code> gets the current date and time.